

Seer Systems

Program Deploy Manage

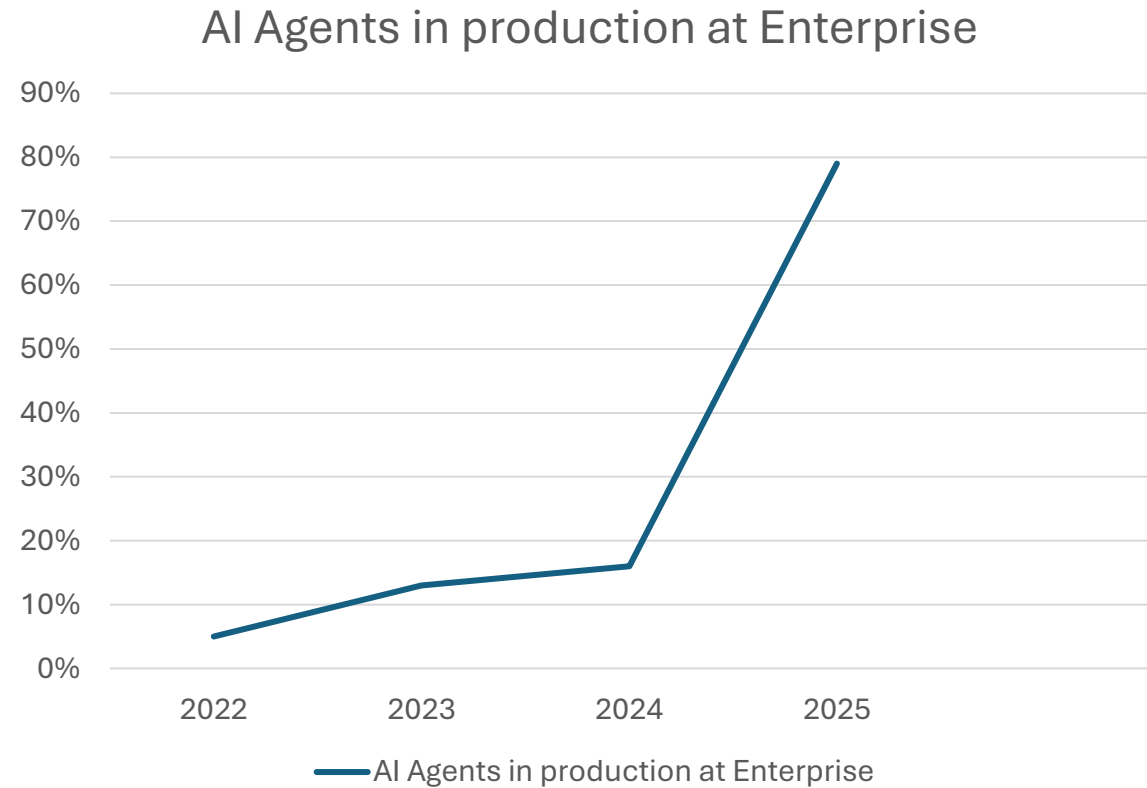
Market Trends

AI agents are getting widespread adoption in the enterprise.

Most companies are deploying a single agent to do complex tasks.

This monolithic architecture leads to various issues, including low reliability, accuracy and performance

To solve this, people are exploring multi-agent systems



Data Source: Langchain

Market Trends

**"A Path Towards Autonomous Machine Intelligence" advocates for a
"society of machines"**

-Yann LeCun, recipient of the Turing Award

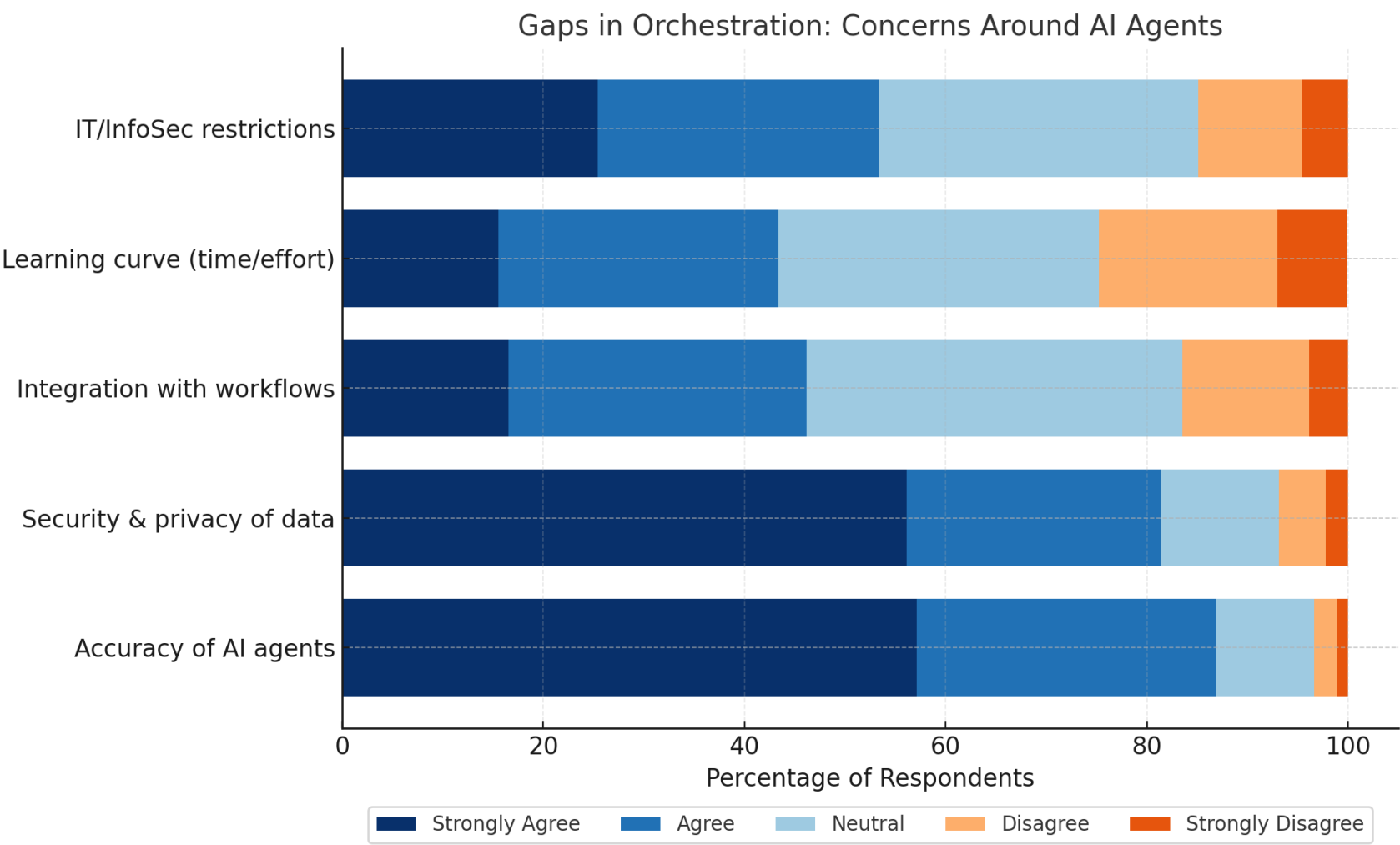
“Agentic AI workflows outperform giant models”

- Andrew Ng, DeepLearning.AI

“Deployments are going to be difficult with multi-agent systems”

-Dario Amodei, Anthropic CEO

The Orchestration Gap



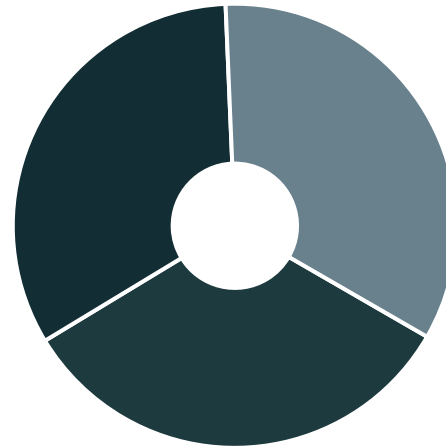
Problem

Complexity & Coordination Overhead

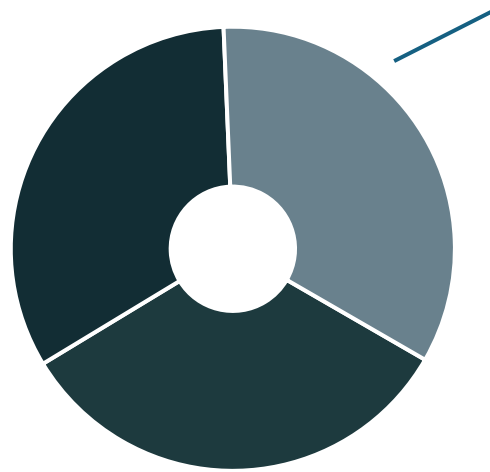
Multi-agent orchestration, whether centralised or distributed, is extremely complex.

Ensuring reliable communication and standardised message-passing between agents is a major pain point

Engineers spend a significant amount of time handling workload distribution and debugging, rather than focusing on agent logic.



Problem



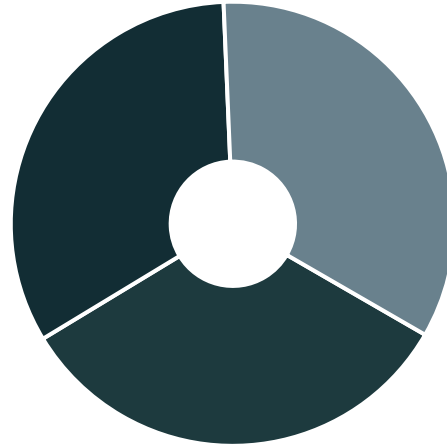
Performance & Resource Constraints

Container runtimes (Kubernetes, Docker) and overlay networking add latency and overhead.

Cold starts, fixed CPU/memory limits, and stateless designs require external storage, introducing additional delays.

Network overhead and container-to-container communication can slow down distributed agents in production.

Problem



Reliability, Observability, Security & Vendor Risks

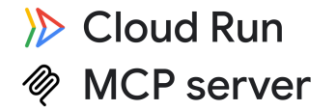
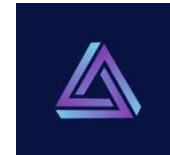
Debugging and monitoring distributed agents is difficult; tracing requests across multiple agents lacks standard tooling.

Shared kernel vulnerabilities, stateless functions, and vendor lock-in create security and portability concerns.

Current tools (LangSmith, AWS Bedrock) help, but are not purpose-built for production-scale agent orchestration.

Today's Frameworks Are Rigid

Today's frameworks, no-code platforms, and protocols are rigid and overly complex, making it painful to spin up and coordinate hundreds of agents



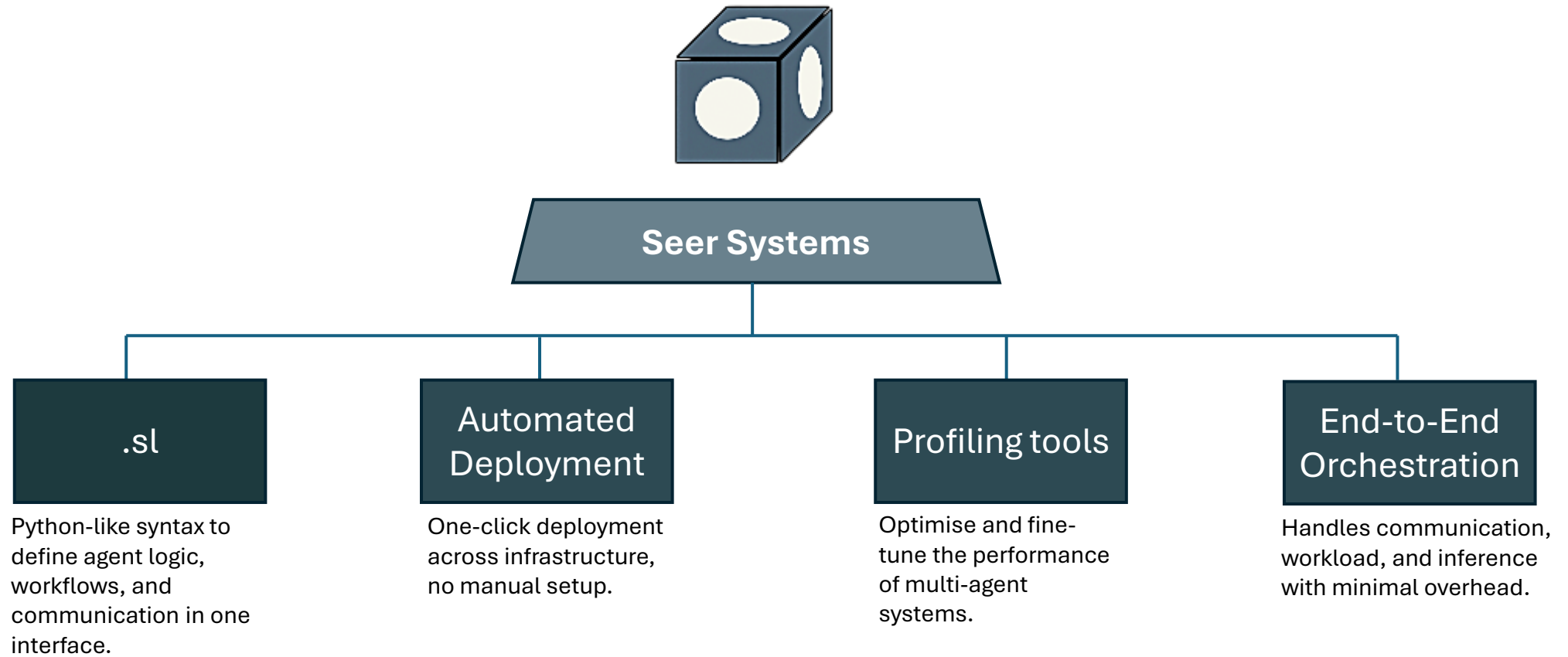
The Solution



Seer Systems

Seer Systems provides a tech stack to program, deploy, and manage multi-agent AI systems

Our solution provides



How does it work?

Step 1

Developers program in .sl

Seer System auto-deploys

Step 2

Developers fine-tune the systems using the profiling tool

```
PIPELINE SocialAgentsChat

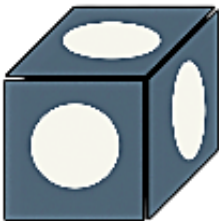
IMPORT
  TOOL LLM          FROM ggenie      USES chat
  TOOL BobMemory    FROM bob_memory  USES get_bob_response, start_rpc_server
  TOOL CarolMemory  FROM carol_memory USES get_carol_response, rpc_call
  TOOL RPC          FROM rpc         USES create_server, create_client, send_message,
                                     receive_message

STATE
  LIST message_history
  STRING last_message
  OBJECT rpc_servers
  OBJECT rpc_clients

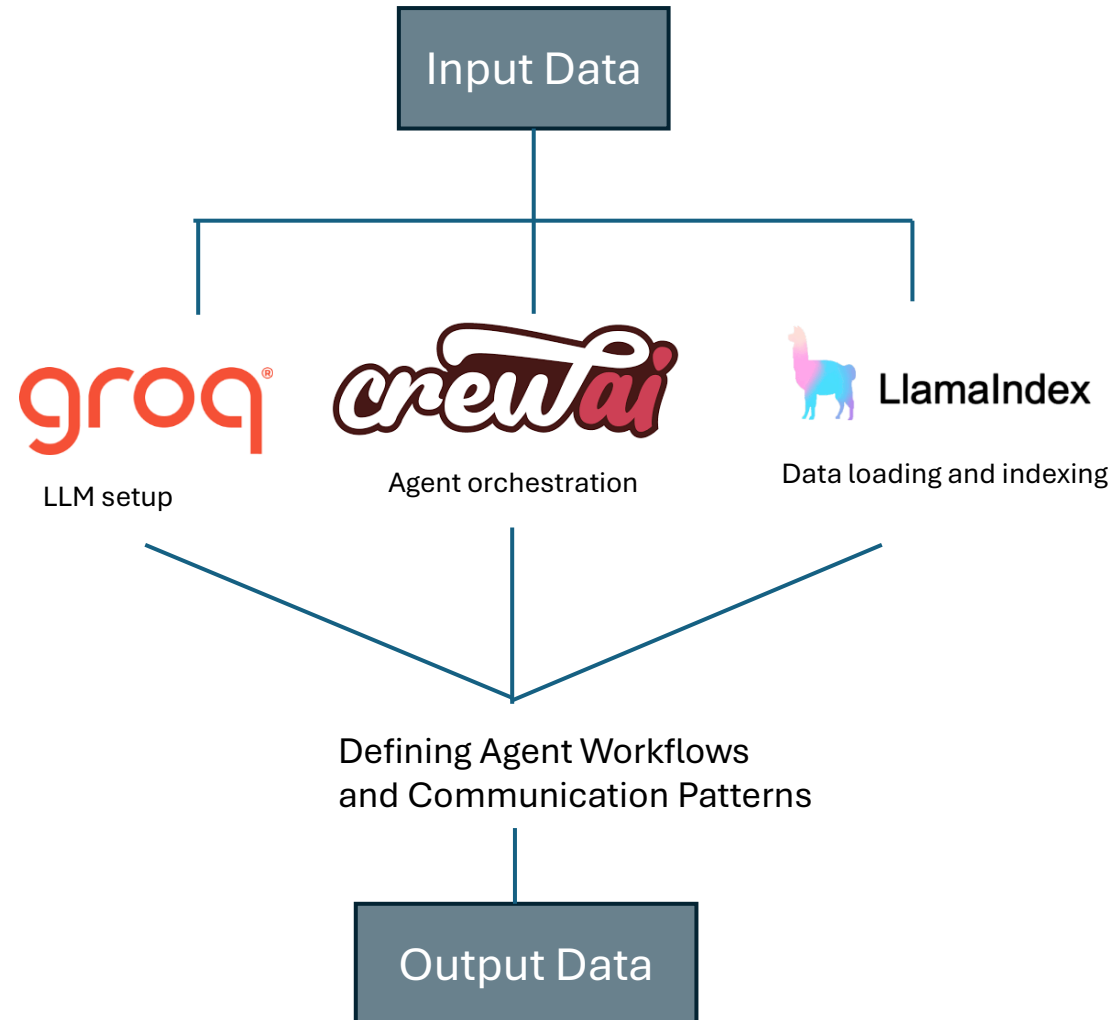
AGENT Alice
  EVENT setup()
    rpc_servers["Alice"] <- RPC.create_server(port=9001)
    rpc_clients["Bob"]   <- RPC.create_client(address="localhost", port=9002)
    rpc_clients["Carol"] <- RPC.create_client(address="localhost", port=9003)

  EVENT onMessageReceived(message)
    SAVE message AS last_message
    APPEND message TO message_history
    TRY
      alice_prompt <- ""

      You are Alice, a language model expert in a social network. You know how to take
      conversations further and behave like an enthusiastic human. You are in a group
      of three friends: Alice, Bob, and Carol.
      Respond to the message sent to you as {message} and keep the conversation going. Keep your
      response precise.
```



Today's Frameworks: Rigid, Painful to Scale, with Logic Entangled in Infrastructure



The Business Cost of Poor Orchestration

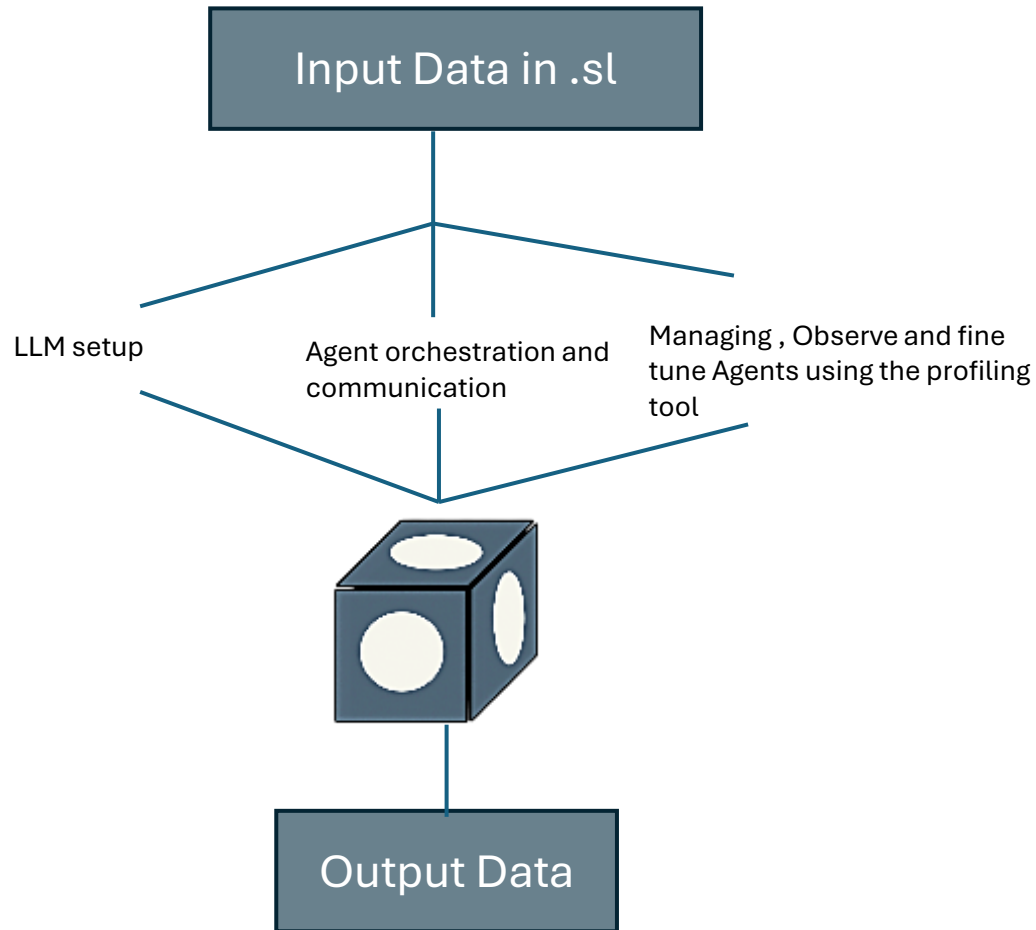
For Developers:

- 30–40% of time lost to custom integration work
- Debugging complexity grows with agent interactions
- Brittle orchestration code creates technical debt

For Businesses:

- Delayed time-to-market
- Rising development + maintenance costs
- Scaling limits block enterprise adoption

Seer Systems handles low-level infrastructure, so developers focus only on agent logic



Orchestration without overhead

For Developers:

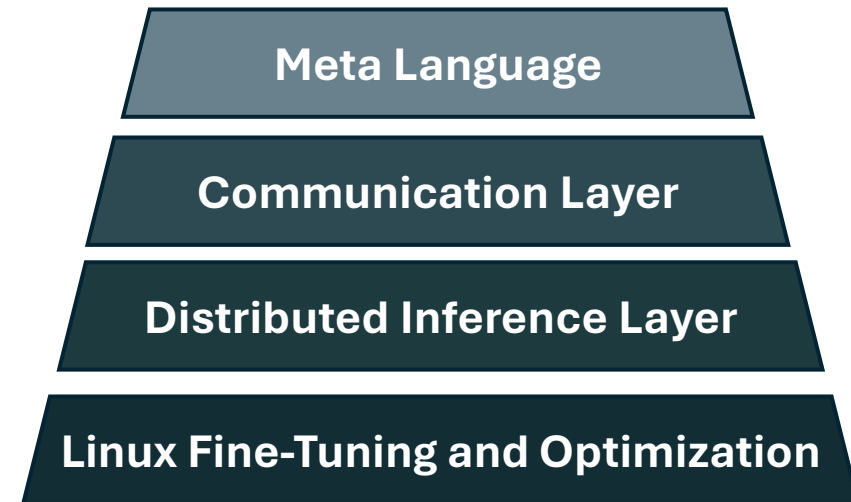
- 100% focus on agent logic, not infrastructure
- Debugging streamlined through clean orchestration
- Low-level abstractions eliminate technical debt

For Businesses:

- Faster time-to-market
- Reduced development + maintenance spend
- Scales seamlessly across hundreds of agents

Technology Stack

What is Seer Systems?



Technology Stack

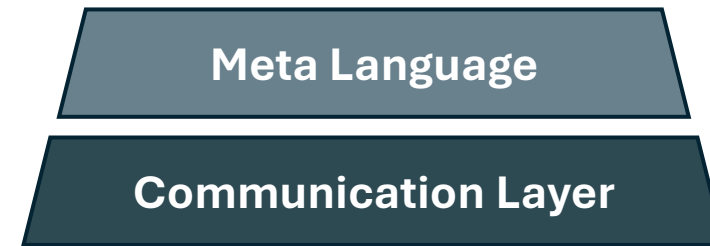
What is Seer Systems?



Meta Language

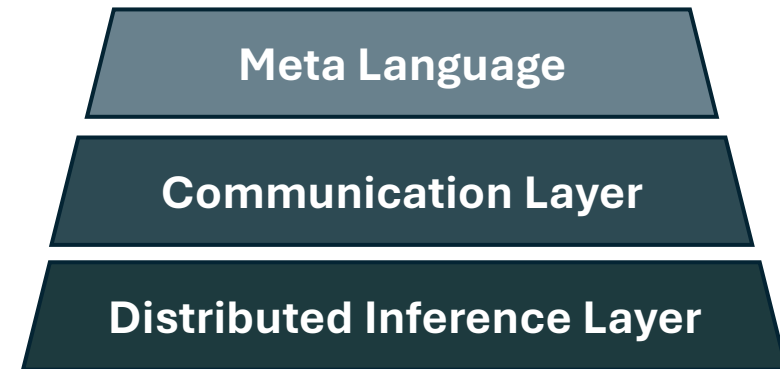
Simplifies agent logic and orchestration with an easy-to-use, Python-like syntax.

What is Seer Systems?



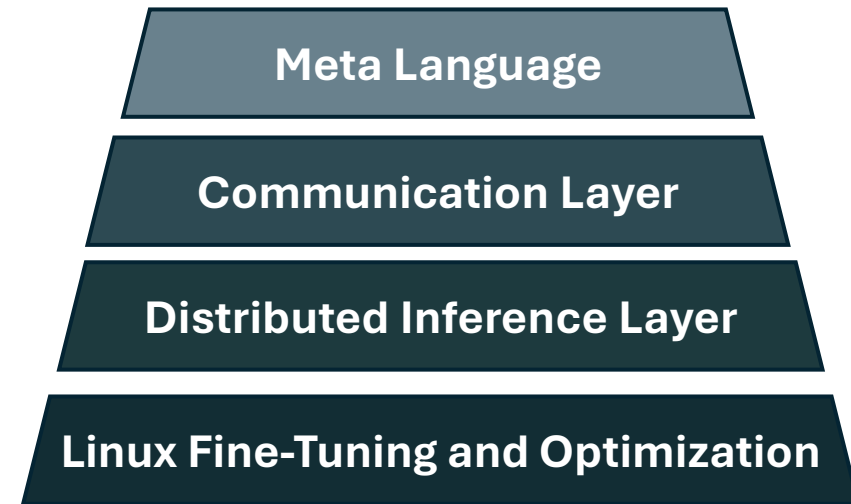
Ensures reliable, standardised agent-to-agent communication over the web, eliminating the need for rigid JSON schema function calling.

What is Seer Systems?



Automates workload distribution across agents, optimizing for scalability and performance.

What is Seer Systems?



Leverages optimized hardware for low-latency, high-throughput performance, minimizing container runtime overhead.

Competitor Analysis

Features	Technologies					
	Llama Index	LangChain	LangGraph	Google ADK	Amazon Bedrock Agents	Seer Systems
	Efficient resource allocation across agents					✓
	Standardized communication channels			✓	✓	✓
	Unified memory and knowledge management	✓	✓	✓	✓	✓
	Transparent execution tracking	✓				
Failure detection and recovery					✓	

Why now?

Ensuring Reliability

Multi-agent checkpointing with verification & validation agents preserves state and prevents failures

Profiling & Accuracy

Our profiling tool tracks accuracy, logs, and agent behavior, enabling fine-tuning for consistent, reliable results

Low Latency Execution

Our communication paradigm reduces latency between agents and ensures smooth execution across distributed systems

Standardized Communication

Standardised messaging enforces consistent interactions and enables reliable scaling across hundreds of agents

Ask

\$500,000 in pre-seed funding

Team



Rajpreet Singh

A post-graduate in High Performance Computing and Systems from TU Munich, Germany. Previously worked at Bell Labs, Qualcomm, TU Munich and Allianz.



Vidhi Kothari

A graduate in Computational Science and Economics from Pace University, New York. Previously worked at Copia Automation, QuadReal Property Group, Newmark, and Morgan Stanley.